

Documento big data

Descrizione del problema (E-commerce)

Il problema che affronterà questo progetto è quello di come progettare e sviluppare un'applicazione che dovrà girare in un sistema distribuito. Questo progetto non riguarda solo come far scalare orizzontalmente un'applicazione ma bensì di come implementare tutte le funzionalità che il business richiede e di come mantenere la consistenza dei dati in un ambiente decentralizzato.

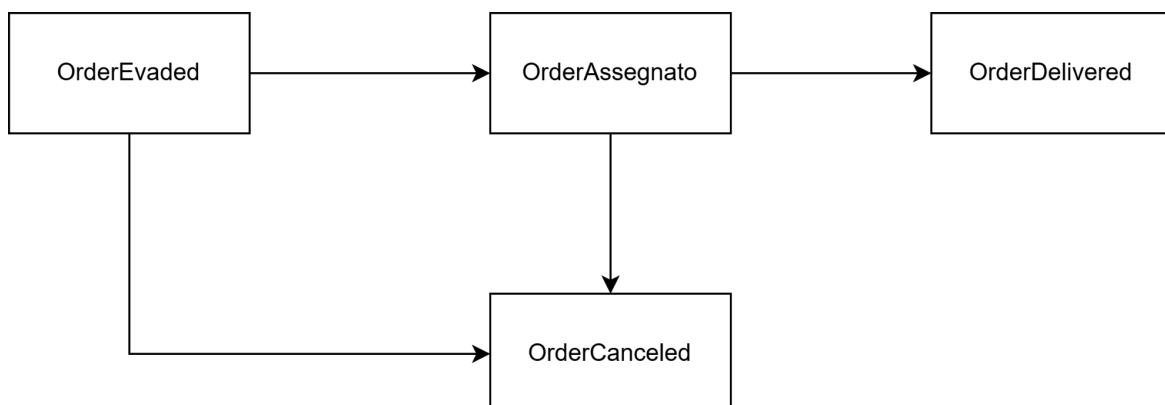
Questo progetto si concentrerà sullo sviluppo di una applicazione di un e-commerce nativa per i sistemi distribuiti che dovrà gestire il ciclo di vita dei prodotti dal loro acquisto fino alla consegna. Questa applicazione coinvolge i seguenti attori:

- **Customer:** Sono le persone che si registrano liberamente nel sito e possono acquistare i prodotti;
- **DeliveryGuy:** i ragazzi delle consegne che hanno l'obiettivo di gestire l'ordine dall'acquisto da parte dei customer fino alla loro consegna o, nel caso di fallimento, fino alla cancellazione;
- **Admin:** Gli amministratori del sistema che possono gestire i vari utenti nel sistema registrando i deliveryGuy o gli altri admin;

I prodotti presenti nel catalogo sono disponibili in quantità limitata e si dovrà garantire che il sistema non possa permettere agli utenti di poter ordinare più prodotti di quanto disponibili.

Gli ordini una volta evasi dovranno seguire il seguente ciclo di vita:

- **OrderEvaded:** è il momento in cui un prodotto viene comprato e quindi l'ordine viene creato;
- **OrderAssegnato:** è il momento in cui il sistema assegna l'ordine ad un deliveryGuy;
- **OrderCanceled:** significa che l'ordine ha avuto un qualche problema e quindi è stato eliminato
- **OrderDelivered:** accade quando il deliveryGuy consegna l'ordine al Customer;

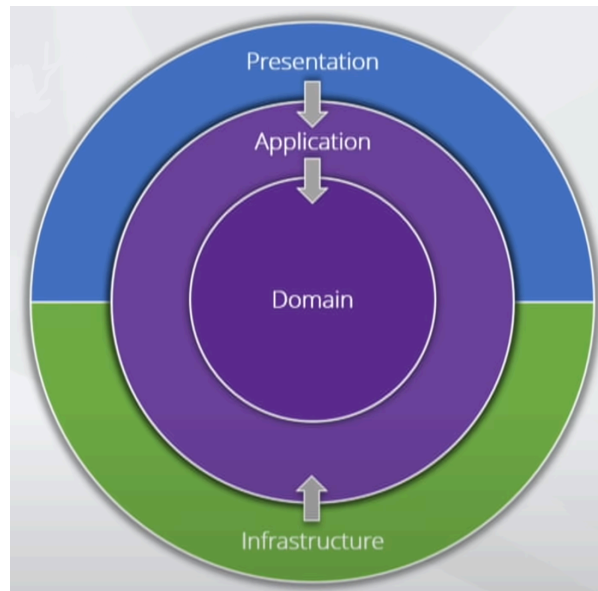


Una volta che un ordine viene evaso il sistema lo assegna al deliveryGuy che ha meno ordini da gestire in quel momento.

Architettura del software

L'architettura proposta per realizzare questo sistema si basa molto sui principi della Onion Architecture, che descrive il software come come serie di cerchi concentrici dove:

- **Nel layer centrale:** abbiamo il dominio cioè la rappresentazione di tutte le entità e le business rule che il sistema dovrà implementare
- **Application layer:** Qui vengono implementate le funzionalità effettive dell'applicazione
- **Infrastructure / Presentation:** E' lo strato più esterno che gestisce i dettagli tecnici del sistema, occupandosi sia della comunicazione con l'utente (Presentation) sia della persistenza dei dati e dei servizi esterni (Infrastructure).



Dominio

La onion architecture ci dice che il dominio (il cuore del sistema) dovrebbe essere completamente indipendente da tutto il resto, nella pratica però questa cosa è praticamente impossibile. Infatti se noi scrivessimo nel cuore del nostro software delle entità pesantissime, ad esempio scrivessimo che l'entità utente deve contenere tutta la lista dei suoi ordini, la quale può aumentare potenzialmente all'infinito, diventerebbe impossibile a livello di infrastruttura scalare.

Domain driven design (DDD)

Per risolvere il problema del dominio ed evitare che diventi troppo complesso ho adottato il principio del domain driven design. Questo principio non solo ci permette di rappresentare il dominio più semplicemente ma si sposa perfettamente con l'architettura distribuita del sistema. I principi DDD che sono stati usati in questo progetto sono:

- **Entità:** Oggetti definiti da un'identità univoca, praticamente uguale al concetto di entità nello schema ER.
- **Value Object:** Oggetti che non hanno identità e sono definiti solo dal valore dei loro attributi e pertanto sono immutabili.
- **Aggregate:** Insieme di entità e value object che sono trattati come un'unica unità per le modifiche. A volte coincidono con una entità
- **Domain event:** Eventi che il dominio (di solito gli aggregati) usa per notificare le varie parti del sistema.
- **Bounded context:** è il confine logico che divide le varie parti dell'applicazione. Un po' come i dipartimenti di una azienda.

Questi concetti sono stati determinanti nella definizione dell'architettura. In particolare, l'adozione degli Aggregate ha permesso di individuare con precisione i dati necessari per eseguire le diverse operazioni di business in modo strettamente consistente. Grazie a questa scomposizione, è stato possibile determinare fin dalle prime fasi di progettazione l'esatto perimetro transazionale delle query al database. Un aggregate di solito riguarda una o più righe in un database relazionale oppure un documento in un database document based.

I Domain event invece, hanno permesso di disaccoppiare la logica del dominio per fare in modo che il dominio non debba gestire l'intera transazione in modo sincrono, permettendo così di implementare un modello di consistenza del tipo Eventual consistency.

I Bounded context invece, hanno permesso di dividere in aree la nostra applicazione permettendo di capire fin da subito qual era la suddivisione migliore dei microservizi.

Traduzione nella nostra applicazione

I bounded context sono 3:

- **Gestione degli utenti:** Questo contesto ha generato il servizio UserService il quale contiene i seguenti aggregate:
 - Admin: Rappresenta gli utenti degli amministratori di sistema
 - Customer: Rappresenta gli utenti che faranno acquisti
 - DeliveryGuy: Rappresenta gli utenti dei ragazzi delle consegne
- **Gestione dei prodotti:** Questo contesto ha generato il servizio ProductService il quale contiene i seguenti aggregate:

- Prodotto: Rappresenta un prodotto nel sistema con la relativa quantità
- Acquisto: Rappresenta l'acquisto effettuato dal Customer
- Gestione delle consegne: Questo contesto ha generato il servizio DeliveryService il quale contiene i seguenti aggregate:
 - DeliveryGuy: Rappresenta quanti ordini sta seguendo un ragazzo delle consegne

Da notare che DeliveryGuy è stato inserito in 2 bounded context diversi. La ragione è che DeliveryGuy rappresenta 2 concetti diversi in 2 parti del sistema diverse, infatti, nella gestione degli utenti rappresenta proprio un Utente quindi con Username, password ecc... mentre nella gestione delle consegna rappresenta proprio il ragazzo delle consegne con il numero di ordini che gli è stato assegnato.

Applicare DDD ci ha permesso quindi di dividere l'applicazione non in base solo ad esigenze tecniche ma in base anche a com'è la realtà che il sistema dovrà riprodurre riducendo i rischi legati a dover implementare delle feature che stanno troppo "in mezzo" a due servizi.

Over-engineering

Questa soluzione proposta di rappresentare il dominio potrebbe sembrare troppo costosa e inutilmente complessa. Ovviamente bisogna valutare progetti caso per caso però se sappiamo che un progetto dovrà essere caricato nel cloud e aperto ad un pubblico di persone piuttosto ampio, questo approccio ci permette di scrivere la logica già in modo scalabile e ben organizzata. Inoltre questo approccio riguarda solo il dominio, poi non si è per forza obbligati a dividere subito in micro-servizi e implementare degli eventi asincroni, volendo si può caricare tutto nello stesso server e gli eventi risolverli dentro la stessa transazione, di conseguenza, andare poi a scalare dove sarà necessario e quando sarà necessario.

Application layer

L'application layer è dove verranno implementate le effettive funzionalità del sistema, di solito corrisponde alle api REST di un singolo servizio.

Le funzionalità implementate dentro a questo layer di solito si dividono in 2 categorie:

- **Query:** delle funzionalità che permettono all'utente di vedere lo status del sistema senza alterarlo.
- **Comandi:** delle funzionalità che provocano un cambiamento di stato del sistema, di solito sono racchiusi dentro ad una transazione. I comandi di solito hanno lo scopo di prelevare l'aggregate da un repository, applicare i cambiamenti e salvare questi cambiamenti sempre tramite repository. Nel caso la funzionalità richiedesse di avere più di un aggregatore l'Application

layer sarà incaricato di coordinare i vari aggregate.

Infrastructure layer

Questo layer è incaricato di gestire tutti i particolari software per accedere all'infrastruttura. Nel nostro caso specifico sono stati implementati i vari repository per accedere al database e l'implementazione della pubblicazione degli eventi e ricezione degli eventi.

Repository

I repository sono fondamentali per astrarre il metodo con cui l'applicazione accede ai dati. In un contesto come il cloud è fondamentale visto che da un momento all'altro si potrebbe passare da un servizio che implementa il database ad un altro. Inoltre centralizzare le chiamate al database ci permette in futuro di implementare dei layer di cache.

Gestione degli eventi

I domain events, generati all'interno del Dominio, vengono intercettati e gestiti dall'Infrastructure Layer nel momento in cui l'applicazione esegue il commit della transazione.

La gestione degli eventi è divisa in 2 moduli installati su ogni servizio:

- **Un publisher:** che ha il compito di prendere gli eventi memorizzati nel database e di inserirli nella coda
- **Un consumer:** che ha il compito di prendere gli eventi e di inoltrarli all'event handler corretto

Dual Write problem

Questo problema accade nel momento in cui io voglio salvare i dati in un database e allo stesso tempo ad esempio, inserire i messaggi in una coda. Visto che le due operazioni non possono essere eseguite in modo atomico, perché se io invio i dati nella coda e la transazione fallisce al commit ormai i dati non li posso tirare fuori ma, allo stesso tempo non posso farlo dopo perché se è l'invio degli eventi a fallire io perderò gli eventi. La soluzione che di solito si implementa è quella di creare una tabella nel database per contenere tutti gli eventi che devono essere spediti, serializzati in un campo json per maggiore flessibilità. Al momento del commit si salverà all'interno della transazione anche i vari eventi e, una volta salvati, ci penserà un processo esterno a scaricare gli eventi dal database e inviarli nella coda. Questo approccio di solito è considerato il miglior trade off perché è vero che l'invio nella coda potrebbe ancora fallire, però almeno hai gli eventi ancora memorizzati nel database e quindi puoi fare un retry.

Architettura del sistema

Il sistema è costituito da 5 moduli principali che sono distribuiti come microservizi nei pods nell'ecosistema di kubernetes.

User service

Lo user service è il modulo che permette all'applicazione di gestire gli account del sistema. Questo modulo permette la registrazione (libera) di nuovi Customer e permette agli admin di creare nuovi customer, delivery guy e altri admin.

Autenticazione

Nei sistemi monolitici l'autenticazione è centralizzata e si basa su sessioni, dove il server genera e memorizza un identificativo univoco al momento del login il quale verrà spedito all'utente tramite cookie. Tuttavia, questo approccio non è possibile in una architettura a microservizi per i seguenti motivi:

- **Il servizio diventa stateful**, in un server monolito non è un problema ma nell'ecosistema kubernetes sì, in quanto qualunque pod può essere cancellato / replicato in qualunque momento. In pratica i pods per poter essere replicati devono essere stateless.
- **Collo di bottiglia** dello user-service, se lo user-service contenesse lui i dettagli delle sessioni allora ad ogni richiesta che deve essere autenticata dagli altri servizi passerebbe per forza prima dallo user-service.

Cookie JWT

La soluzione proposta a questi problemi è l'utilizzo dei token jwt all'interno dei cookie.

Un token jwt è come una stringa di testo divisa in 3 parti separate da un punto (.):

- **Header**: che contiene le informazioni tecniche sul token come l'algoritmo di firma digitale utilizzato
- **Payload**: contiene i dati della sessione necessari ai sistemi di autenticazione come ad esempio l'id utente e il suo ruolo.
- **Firma**: questa parte contiene la firma digitale del token generata direttamente dalla chiave privata del server. Questa firma è fondamentale per fare in modo che i dati del payload non possano essere alterati.

Questo token permette l'autenticazione dell'utente pur tenendo i servizi stateless. In pratica la sessione viene estratta ad ogni richiesta http dal token jwt.

Sicurezza token JWT

Questo token viene salvato nella sezione cookie del browser in modo tale che non sia accessibile dal javascript (E una buona pratica per evitare che estensioni

malevole possano rubare il cookie). Però per essere effettivamente sicuro bisogna fare in modo di gestire i flag del cookie in modo corretto:

- **HttpOnly**: il cookie deve avere questo flag per non essere visibile al javascript
- **Secure**: impedisce di inviare il cookie su connessioni HTTP non protette
- **SameSite**: Questo flag deve essere impostato su strict o lax per bloccare l'invio automatico dei cookie su siti terzi.

Endpoints

Url	Descrizione
POST Utente/Login	Autentica l'utente tramite username e password rilasciando un token JWT salvato in un cookie HttpOnly
POST Utente/Logout	Invalida la sessione dell'utente cancellando il cookie di autenticazione tramite l'impostazione di una data di scadenza passata.
GET Utente/UtenteLoggato	Restituisce i dati identificativi, lo username e il ruolo dell'utente attualmente loggato.
GET Utente	Permette esclusivamente agli admin di recuperare la lista completa di tutti gli utenti registrati nel sistema con le relative generalità.
POST Utente	Consente a un admin di creare un nuovo utente nel sistema specificando il ruolo tra Admin, Customer o DeliveryGuy
POST Utente/Registrazione	Permette l'auto-registrazione pubblica di un nuovo utente nel sistema impostando automaticamente il ruolo predefinito di tipo Customer

Eventi emessi

Nome	Descrizione
DeliveryGuyCreatoEvent	Accade quando viene registrato un nuovo delivery guy nel sistema

Product service

Questo servizio è responsabile di gestire il catalogo dei prodotti e le loro quantità. Fa in modo di gestire gli acquisti degli utenti controllando che non si possa ordinare più quantità di prodotto di quella che è effettivamente disponibile.

Endpoints

Url	Descrizione
GET Prodotti	Recupera la lista completa di tutti i prodotti ordinati per nome, mostrandone il codice, il nome e la quantità disponibile.
POST Prodotti	Permette esclusivamente agli admin di inserire un nuovo prodotto nel catalogo specificando nome e quantità iniziali.
POST Prodotti/{id}/Acquista	Consente ai soli clienti di acquistare una determinata quantità di un prodotto.

Eventi emessi

Nome	Descrizione
AcquistoCreatoEvent	Accade quando viene effettuato l'acquisto di un prodotto

Delivery service

Questo servizio gestisce l'intero ciclo di vita di un ordine tracciandone gli stati, da quando viene creato fino alla sua conclusione. Una volta che un ordine viene creato esso viene assegnato al corriere con meno ordini attivi. Infine, registra la chiusura formale dell'ordine gestendo sia il successo della consegna sia l'eventuale cancellazione.

Endpoints

Url	Descrizione
GET Order	Permette ai DeliveryGuy di visualizzare la lista dei propri ordini assegnati, ordinati per momento di acquisto e completi di dettagli logistici.
POST Order/{id}/Consegna	Consente al corriere assegnato di registrare l'avvenuta consegna di uno specifico ordine
POST Order/{id}/Cancella	Consente al corriere assegnato di annullare uno specifico ordine

Eventi emessi

Nome	Descrizione
OrderCreatoEvent	Si verifica quando un nuovo ordine viene creato
OrderCompletatoEvent	Si verifica quando un ordine viene consegnato oppure viene cancellato

Consumers

Nome	Descrizione
AcquistoCreatoConsumer	Una volta che viene effettuato un acquisto il sistema crea un ordine
OrderCreatoConsumer	Una volta che l'ordine viene creato gli viene assegnato un deliveryguy
DeliveryGuyCreatoConsumer	Quando il sistema registra un nuovo delivery guy il sistema il delivery service lo registra per potergli assegnare le consegne

OrderCompletatoConsumer	Quando un ordine viene completato il sistema segnala che il delivery guy ha un ordine in meno attivo
-------------------------	--

Frontend-Web

Il front-end web viene distribuito tramite Nginx, un server HTTP leggero, inverso e altamente configurabile. Poiché l'applicazione è sviluppata in Blazor WebAssembly, Microsoft fornisce un file di configurazione Nginx predefinito, ottimizzato specificamente per gestire l'applicazione come Single Page Application (SPA) e risolvere i problemi di routing interni.

Kubernetes

Kubernetes è un orchestratore automatizzato che coordina le applicazioni distribuite tramite immagini docker all'interno del cluster. Nel nostro sistema viene utilizzato come piattaforma per far girare tutti i nostri servizi, infatti ogni microservizio è un pod. Kubernetes ci permette di aumentare e diminuire il numero di repliche dei nostri servizi in base al carico del sistema in un determinato istante rendendolo lo strumento ideale per progettare un'applicazione che deve scalare orizzontalmente. Kubernetes inoltre permette di effettuare upgrade del software con zero downtime del servizio.

Kubernetes utilizza i Deployment per gestire il ciclo di vita e il deployment dei singoli servizi, e i Service come layer di astrazione di rete per consentire l'instradamento del traffico e l'interconnessione tra di essi. L'accesso dall'esterno verso questi servizi è regolato e instradato da un componente di Ingress

per vedere le configurazioni specifiche vedere i files nella cartella: kubernetes

YugabyteDB

YugabyteDB è un database sql progettato per combinare la possibilità di fare transazioni ACID e tutte le funzionalità dei database relazionali con la scalabilità orizzontale tipica dei database NoSQL.

Dal punto di vista dell'applicazione il database è un unico database logico PostgreSQL, ma in realtà viene suddiviso in più nodi.

La scelta di adottare questo database all'interno del nostro sistema è dovuta al supporto nativo per kubernetes grazie anche al suo operator ufficiale. Un database tradizionale pensato per girare su un solo nodo avrebbe difficoltà a girare in kubernetes e molto probabilmente richiederebbe un servizio esterno. Yugabyte invece sfrutta proprio kubernetes per gestire automaticamente i suoi nodi.

Tablet

In Yugabyte una query viene divisa in shard chiamati tablet e rappresentano l'unità fondamentale di partizionamento. Quando viene creata una tabella questa viene automaticamente divisa in più shard tramite la chiave primaria facendo un hash oppure in base a dei range. I tablet di solito vengono distribuiti su diversi nodi YB-TServer per ripartire il carico

DocDB

DocDB è il motore di archiviazione, rappresenta il cuore tecnologico di ogni YB-TServer ed è responsabile della persistenza fisica dei dati su disco e della loro replica. Questo motore permette inoltre di gestire le transazioni ACID con livelli di isolamento Snapshot e serializable

Raft

Raft è l'algoritmo di consenso distribuito utilizzato per garantire che tutti i nodi di un cluster concordino sullo stesso stato dei dati, assicurando tolleranza ai guasti e consistenza. In pratica le table vengono replicate su più nodi eleggendo uno come leader. Le scritture avvengono nel leader e poi documentate all'interno di un log così che anche le repliche possono sincronizzarsi. Nel caso il leader abbia un guasto un replica prende il suo posto.

YB-Master

Il servizio YB-Master è il componente che gestisce il control plane (piano di controllo) di un cluster YugabyteDB. Non si occupa di elaborare le letture o le scritture dei dati degli utenti, ma coordina l'intero stato e la stabilità del database. Le sue funzioni principali sono:

- **Gestione dei metadati:** Conserva le informazioni strutturali del sistema come la definizione delle tabelle.
- **Tracciamento dei Tablet:** Monitora la posizione dei tablet distribuiti sui vari nodi YB-TServer
- **Bilanciamento:** Rileva i guasti dei nodi e gestisce la replica automatica dei dati mancanti e gestisce il load-balancing

YB-TServer (Tablet server)

Il servizio YB-TServer gestisce il data plane (piano dei dati) di un cluster YugabyteDB. È il componente che si occupa di ospitare fisicamente i dati e di elaborare tutte le richieste di lettura e scrittura degli utenti. Le sue funzioni principali sono:

- **Gestione dei tablet:** Ogni tabella viene divisa in frammenti chiamati tablet

- **Motore di archiviazione:** Utilizza internamente DocDB, un motore di memorizzazione che permette di gestire dati distribuiti e transazioni acid.
- **Esecuzione delle query:** Contiene i layer api compatibili sia per Postgresql e per cassandra per tradurre e ottimizzare le query dei client in operazioni sui tablet

RabbitMQ

RabbitMQ è un message broker che permette a diversi microservizi di comunicare tra loro tramite lo scambio di messaggi che nel nostro caso sono gli eventi di dominio.

Questo broker si frappone in mezzo ai microservizi gestendo automaticamente l'invio, la ricezione e la persistenza dei messaggi.

Il publisher

I publisher sono le applicazioni che generano i messaggi e gli inviano al broker che si chiama Exchange

Exchange

È il punto di ingresso dei messaggi della coda. Una volta ricevuto un messaggio questo modulo lo instrada tramite la routing key, una chiave inserita dal publisher, e tramite le regole di binding. Una regola di binding definisce come sono collegati gli exchange e le code dei consumer, in pratica, definisce quali messaggi in base alla routing key si è interessati a ricevere.

Consumer

I consumer sono quelli che vogliono ricevere i messaggi e per farlo li leggono dalla coda a loro associata. Essendo che l'elaborazione del messaggio potrebbe causare un'eccezione la coda ha un meccanismo di acknowledgement grazie al quale elimina il messaggio solo nel momento in cui gli viene segnalato come processato.

Implementazione della nostra applicazione

Ogni microservizio dentro alla nostra applicazione può sia emettere eventi sia riceverli. Per questa ragione è stato implementato un meccanismo automatico che alla ricezione di un messaggio il servizio automaticamente va a trovarsi gli handler corretti.

Per vedere i dettagli implementativi basta vedere i file con tutti i parametri commentati: OrderManagementCommon/Workers

Idempotenza degli event handler

L'implementazione della funzione che andrà a gestire l'evento deve essere fatta in modo idempotente cioè, chiamare quella funzione una volta o più volte con lo stesso

input per il sistema deve essere indifferente. Questo è necessario perché rabbitMQ garantisce che il messaggio venga ricevuto almeno una volta ma non esattamente una. Questo significa che potrebbe capitare che lo stesso evento venga elaborato più volte e che quindi bisogna gestire anche questa situazione.

Considerazioni finali sull'architettura

L'architettura implementata permette non solo di scalare in termini di prestazioni ma anche di aumentare il numero di feature che può implementare.

Considerazioni su YugabyteDB

Questo database permette di far scalare orizzontalmente un sistema SQL completamente compatibile con i driver di Postgres. Questo ci permette di risolvere il classico collo di bottiglia dei database relazionali quando si parla di sistemi distribuiti. Yugabyte però mantiene solo l'interfaccia identica a postgresql ma sotto il cofano gestisce, come ad esempio, le transazioni in modo molto differente e "più rozzo" rispetto a postgresql. Ad esempio postgresql permette di avere i predicate lock per avere i lock su una tabella, durante una transazione con livello di isolamento serialization, permettendo di gestire molto meglio più transazioni in parallelo. Yugabyte invece non ha questa tecnica, visto che in contesto distribuito sarebbe impossibile metterla in pratica, e di solito quando il livello di isolamento è serializable il lock è messo a livello di tabella facendo abortire automaticamente molte transazioni. Questo significa che è vero che l'interfaccia è la stessa però questo tipo di database richiede molto più lavoro lato applicativo per gestire il carico concorrente.

Considerazioni su kubernetes

Implementare subito il sistema utilizzando kubernetes potrebbe essere un costo eccessivo specie per le prime fasi di un progetto. I costi sono sia in termini di complessità del progetto sia in termini di costo da pagare di infrastruttura cloud, pertanto sarebbe meglio prima partire con docker e docker-compose e poi successivamente se sarà necessario migrare verso kubernetes. Questa migrazione non richiede una riscrittura completa del software o dell'intero sistema perché il nostro software è già stato progettato per lavorare dentro ai container pertanto è "solo" un cambio di infrastruttura.